

## **Reporte de Proyecto**

### **Definición del Problema**

El problema radica en crear un sistema capaz de reconocer la temperatura ambiental y a partir de esta responder dependiendo de si es muy alta o muy baja según los parámetros. Ser capaz de regular la temperatura de un objeto es importante, por ejemplo, la temperatura de una computadora, la temperatura de una habitación o de algún dispositivo. Este problema puede surgir en cualquier lado, por lo que se necesita crear un sistema capaz de cambiar la temperatura esperada dependiendo de donde se encuentre. Para centrar el uso, se llevará a cabo específicamente en la Universidad, en un aula con alrededor de 15 estudiantes presentes, con y sin aire acondicionado y distintas variables que pueden afectar la temperatura. La implementación de un sistema inteligente es importante ya que, pese a que existen instrumentos mecánicos que indican la temperatura la tecnología facilita realizar avisos mediante LEDs o bocinas. La tecnología también facilita indicar a partir de qué temperatura se quiere realizar el aviso. Al automatizar estos procesos, no es necesario una persona que esté constantemente revisando la temperatura que se desea medir.

### **Solución propuesta al Problema**

Para encontrar una solución, se propone un sistema inteligente de monitoreo de temperatura, el cual mide en tiempo real la temperatura usando un sensor digital y una pantalla LCD.

Cuando la temperatura se encuentra dentro de los parámetros definidos, el sistema permanece en estado normal con la luz LED encendida de color verde. Si la temperatura se sale del rango establecido, ya sea por encima o debajo de lo esperado, se prende una alerta conformada por el LED y la bocina, notificando al usuario que se encontró una temperatura indebida.. De igual forma, si el sistema detecta una temperatura muy caliente, enciende un ventilador(motor) para enfriar el ambiente mientras va mostrando los valores en la pantalla LCD.

El sistema utiliza diferentes elementos electrónicos que permiten el funcionamiento del mismo, tales como:

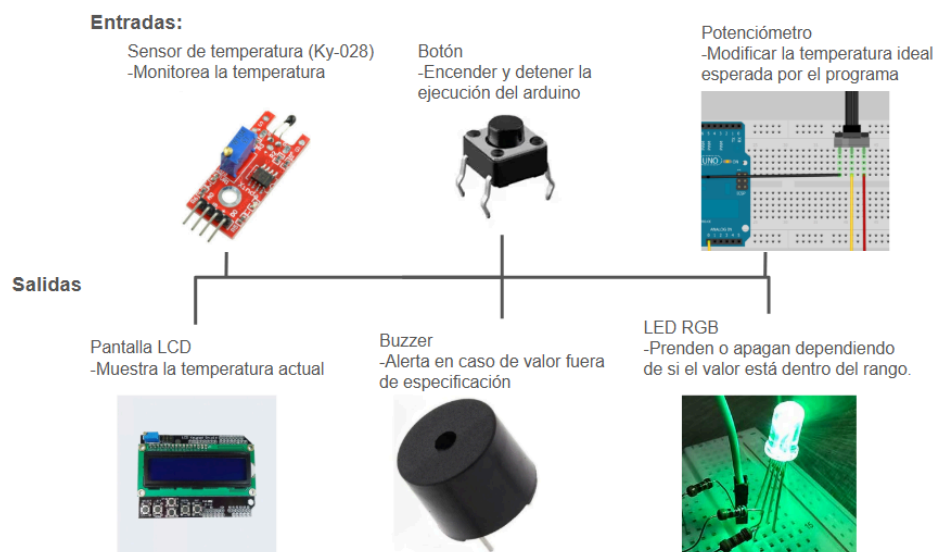
- Sensor de temperatura: tener un control de la temperatura.
- Pantalla LCD: Permite al usuario ver en tiempo real los valores registrados.
- Motor: ayudar a regular la temperatura (simula un ventilador)
- LED: proporciona una señal visual para identificar el estado del sistema.
- Buzzer: genera una alerta sonora en caso de valores fuera de rango
- Potenciómetro: cambiar la temperatura esperada mientras el programa está en ejecución.

Para la solución, se usará un Arduino ya que con él se puede integrar sensores y circuitos. Además su programación facilita la creación de proyectos y tiene una buena compatibilidad con módulos como pantallas LCD, LEDs y buzzers.

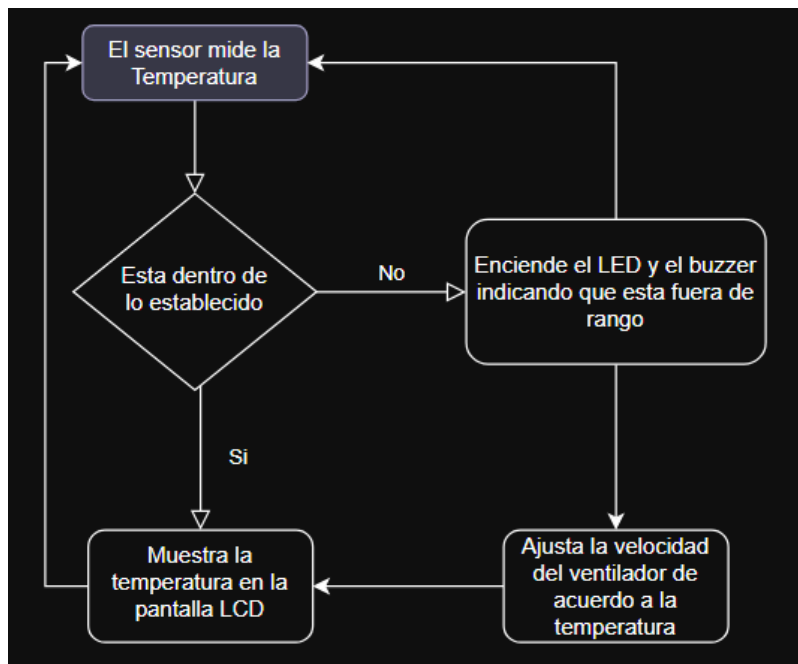
## Lista de componentes electrónicos a utilizar

- Arduino, Resistencias, cables de conexión, protoboard y fuente de alimentación.
- Sensor de temperatura.
- Pantalla LCD.
- LED (rgb).
- Buzzer.
- Botón
- Buzzer
- Motor
- Potenciómetro

## Diagrama de Entradas/Salidas:



## Diagrama de estados



## Introducción

Para la realización de este sistema se utilizará la plataforma de Arduino UNO R3. UNO R3 es una placa microcontroladora perteneciente a la gama de placas UNO, la más utilizada de la familia Arduino. La UNO R3 contiene un chip ATmega382P, un chip de la compañía Atmeg basado en arquitectura RISC con 1 kb de memoria EEPROM que guarda información cuando la placa no posee corriente. Además la placa posee 14 pines para entradas y salidas digitales 6 entradas analógicas, un resonador de cerámica de 16MHz, una conexión USB por medio de USB tipo B, un conector de alimentación, un encabezado ICSP y finalmente un botón para reset. Dicha placa recibe instrucciones por medio código programado en Arduino, una versión simplificada de C++ especialmente diseñada para las placas Arduino, que nos permite controlar cada uno de los componentes de la placa de manera sencilla.

Es importante además para este proyecto entender la diferencia entre componentes digitales y analógicos. Para entender esto debemos primero conocer a qué nos referimos con análogo y digital. Una señal digital es una señal discreta, normalmente con valores 0 o 1, donde dicha señal puede tomar un cierto número de valores. Por otro lado una señal analógica es continua donde cambios como la cantidad de voltaje incrementa o disminuye la señal de manera continua. Por lo tanto componentes digitales y analógicos utilizan este tipo de señales para facilitar ciertos beneficios como puede ser en nuestro caso un sensor digital de temperatura, que al ser digital discretiza el valor de la temperatura facilitando su uso a un nivel donde el error no es tan importante, o múltiples componentes analógicos como luces LED, potenciómetros, o teclados matriciales cuyas salidas o entradas varían de manera continua dependiendo del voltaje (como puede ser la intensidad de la luz en las luces LED).

Con los conceptos y componentes anteriores es posible crear un sistema inteligente con las capacidades deseadas de control y manejo de temperatura. Dicho sistema se trata automatiza una serie de procesos, en este caso la medición de una temperatura óptima, por medio de la placa Arduino UNO R3, los componentes digitales y analógicos y el uso de la programación para crear un sistema inteligente que capaz de cumplir la tarea de manera automática tras su configuración.

## Documentación técnica de los componentes electrónicos utilizados

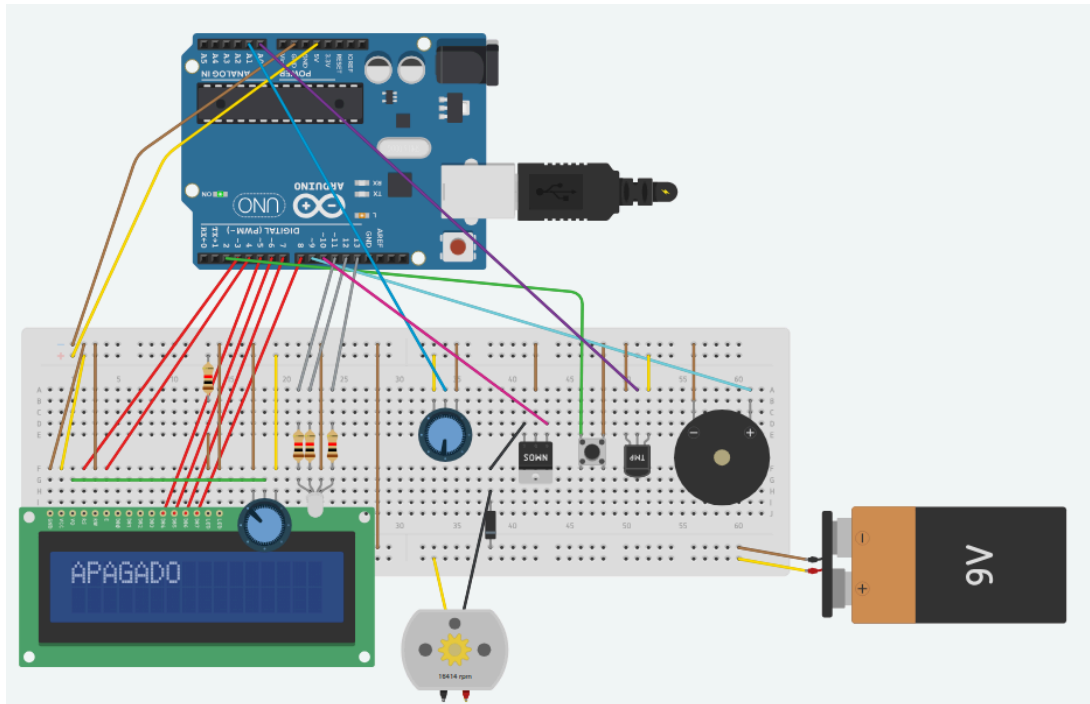
En la primera parte de este segmento del reporte, se va a especificar los modelos de los componentes electrónicos utilizados, seguidamente se describirán las características de la funcionalidad importantes para la aplicación y finalmente cómo se integran estos con la programación en Arduino. Todo esto se realiza con el objetivo de tener una mejor comprensión de todos los componentes utilizados, y mantener un registro de la correcta utilización de estos.

La lista de componentes es la siguiente, seguido de su nombre de modelo exacto.

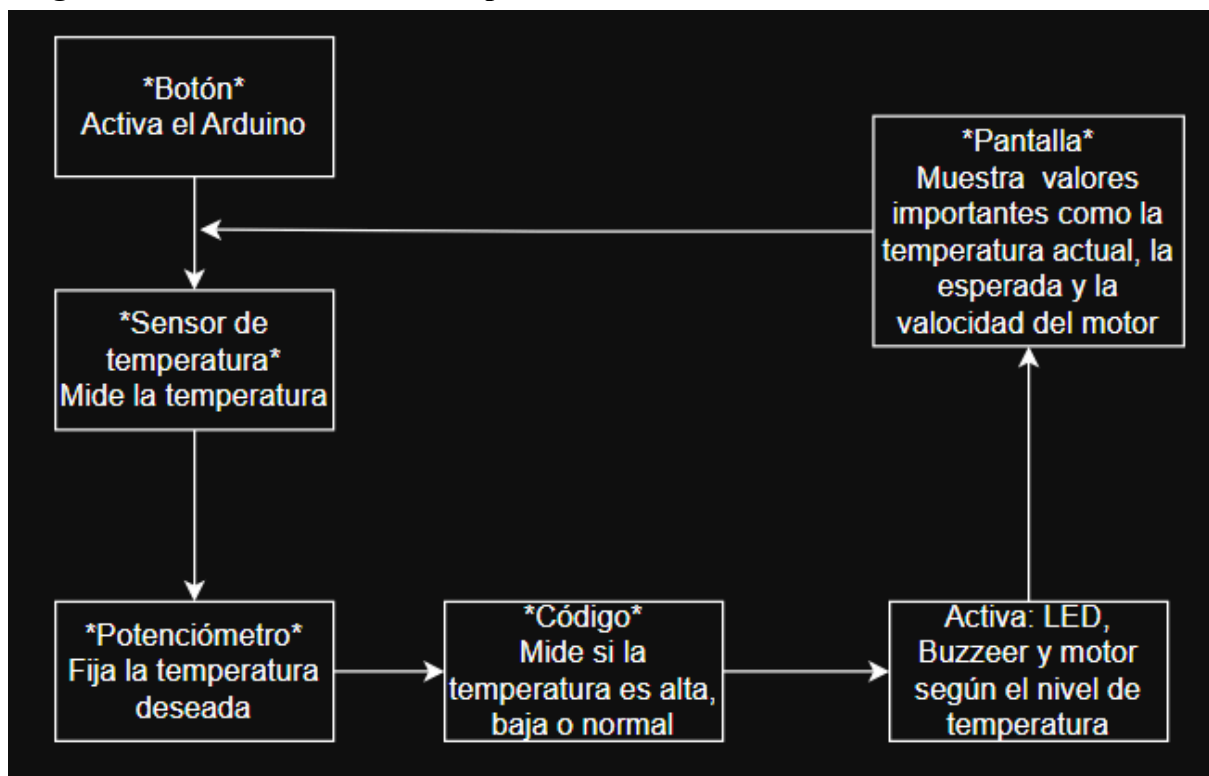
| Componente         | Modelo             | Características                                                                               |
|--------------------|--------------------|-----------------------------------------------------------------------------------------------|
| Pantalla LCD       | JHD659             | 16 caracteres para dos líneas. Gran posibilidad de enseñar hilos de caracteres.               |
| Potenciómetro      | CA-9               | Indica el grado de giro de su eje. Se puede utilizar para la temperatura base en el circuito. |
| Motor              | TFK-280SA-22125    | Torque de 150 g/cm. Importante para simular un ventilador.                                    |
| LED RGB            | YJ503RGB-30B       | Tres LEDs que proporcionan alta posibilidad de colores para indicar temperaturas.             |
| Mosfet             | IRF520NPbF         | Baja resistencia térmica y velocidad rápida a cambio.                                         |
| Switch             | 101-TS6111T1601-EV | Envía una señal para ser interpretada en cualquier contexto deseado.                          |
| Sensor temperatura | KY-028             | Señal analógica que indica la temperatura medida.                                             |

## Diagrama esquemático de la versión funcional

### Versión recreada del arduino en tinkercad



### Diagrama de interacción entre componentes



## Entregable 4: Optimización del programa mediante ensamblador

### Selección del módulo escogida para optimizar:

El módulo seleccionado para optimización es la lectura analógica de sensores, para esto se mejorará la función de `analogRead` específicamente, ya que es de las más viables para mejorar, esta se encarga principalmente de leer la temperatura dada por el sensor.

Actualmente este código se ejecuta dos veces en cada iteración del loop, por lo que mejorarlo es una muy buena acción para mejorar la eficacia del código.

La función anterior utilizada era “`int value = analogRead(pin)`”, pero esta añade retardos innecesarios después de muchas repeticiones, lo que hace que el código sea un poco más lento.

### ¿Por qué ensamblador?

Se eligió ensamblador AVR porque permite:

- Control directo sobre los registros
- Evitando operaciones redundantes.
- Eliminación de ciclos de máquina innecesarios.
- Mejor uso de registros de propósito general.

### Mejoras esperadas

- Reducción del tiempo de lecturas
- Flujo más estable en el control de actuadores.
- Mayor responsividad del sistema al ajustar la temperatura objetivo.

Revisar el primer link, capítulo 23, para especificaciones del `analogRead()`. Es importante entender bien lo de los registros para poder optimizarlos con ensamblador. El `analogRead` originalmente se comporta así.

```
int analogRead(uint8_t pin)
{
    uint8_t low, high;

    if (pin >= 14) pin -= 14; // allow for channel or pin numbers

    // set the analog reference, input pin and clear left-justify (ADLAR)
    ADMUX = (analog_reference << 6) | (pin & 0x07);

    // start the conversion
    sbi(ADCSRA, ADSC);

    // ADSC is cleared when the conversion finishes
    while (bit_is_set(ADCSRA, ADSC));

    // read low and high bytes of result
    low = ADCL;
    high = ADCH;

    // combine the two bytes
    return (high << 8) | low;
}
```

Acá está la versión optimizada con ensamblador.

```
int analogReadAsm(uint8_t pin)
{
    uint8_t low, high;
    asm volatile (
        // Adjust pin number (A0-A7)
        "cpi    %[pin], 14      \n\t"
        "brlo   1f              \n\t"
        "subi    %[pin], 14      \n\t"
        "1:                                     \n\t"

        // Enable ADC and set prescaler (ADEN + ADPS2:0 = 0b111)
        "lds     r18, %[adcsra]  \n\t"
        "ori     r18, 0x87       \n\t" // 1000 0111 = ADEN + prescaler /128
        "sts     %[adcsra], r18  \n\t"

        // Configure ADMUX (REFS0 = AVcc, MUX = pin)
        "ldi     r18, 0x40       \n\t" // REFS0 = 1, REFS1 = 0 (AVcc ref)
        "or      r18, %[pin]     \n\t"
        "sts     %[admux], r18   \n\t"

        // Start conversion
        "lds     r18, %[adcsra]  \n\t"
        "ori     r18, (1<<6)     \n\t" // ADSC = 1
        "sts     %[adcsra], r18  \n\t"

        // Wait for ADSC to clear
        "2:                                     \n\t"
        "lds     r18, %[adcsra]  \n\t"
        "sbrc    r18, 6          \n\t"
        "rjmp    2b              \n\t"

        // Read result (ADCL first, then ADCH)
        "lds     %[low], %[adcl] \n\t"
        "lds     %[high], %[adch] \n\t"
        : [low] "=r" (low), [high] "=r" (high), [pin] "+r" (pin)
        : [admux] "i" (_SFR_MEM_ADDR(ADMUX)),
          [adcsra] "i" (_SFR_MEM_ADDR(ADCSRA)),
          [adcl] "i" (_SFR_MEM_ADDR(ADCL)),
          [adch] "i" (_SFR_MEM_ADDR(ADCH))
        : "r18"
    );

    int result = (high << 8) | low;
    return result;
}
```

La función `analogReadAsm()` realiza una lectura analógica utilizando instrucciones en ensamblador AVR incrustadas dentro de C. Su propósito es obtener un valor ADC de 10 bits

desde un pin analógico del microcontrolador, normalmente en un ATmega328P como el usado en un Arduino Uno.

Primero, la función ajusta el número de pin recibido. Esto es necesario porque en Arduino los pines analógicos A0–A7 están numerados como 14 a 21 en la tabla general de pines. El código compara si el pin es mayor o igual a 14 y, de ser así, le resta 14 para convertirlo en un índice válido entre 0 y 7, que es el rango usado internamente por el ADC.

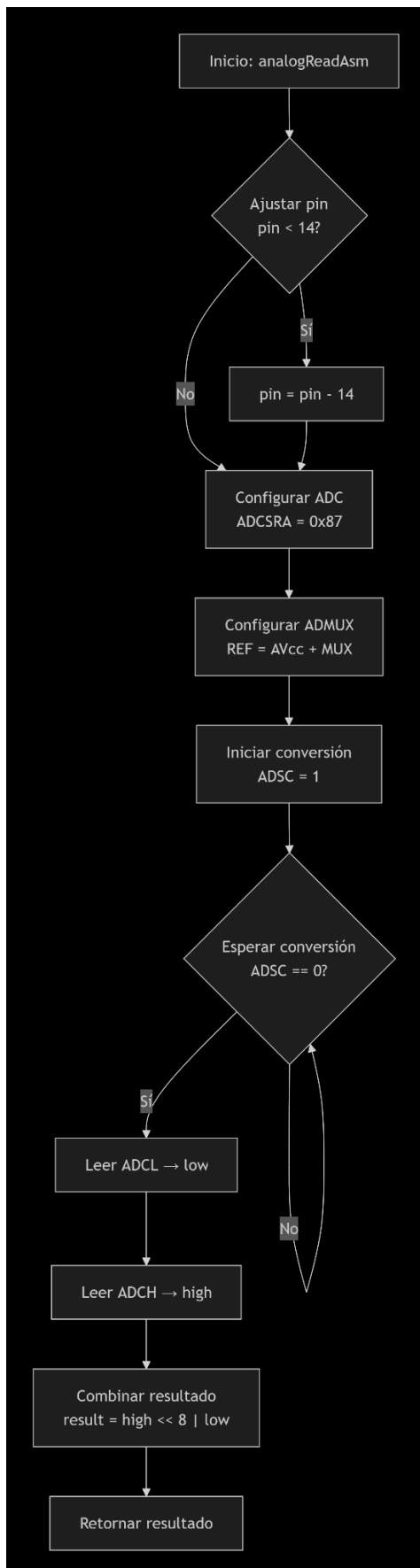
Luego, se configura el registro de control del ADC (ADCSRA). Se lee su valor actual, y se establece el bit ADEN para habilitar el ADC, además de configurar los bits del prescaler (ADPS2:0) con el valor 111, que corresponde a un divisor de reloj de 128. Esto deja el ADC trabajando a unas 125 kHz, frecuencia recomendada para obtener medidas estables y precisas. El siguiente paso es configurar el registro ADMUX, que controla la referencia de voltaje y el canal analógico a leer. Se carga el valor 0x40, que establece la referencia de voltaje en AVcc (usualmente 5 V). Luego este valor se combina con el número del pin ya ajustado, seleccionando así el canal ADC correspondiente antes de escribirlo en ADMUX.

Una vez configurado el ADC, se inicia la conversión. Para esto, el código establece el bit ADSC dentro de ADCSRA. Este bit se mantiene en 1 mientras el ADC está realizando la medición. El programa entra entonces en un pequeño ciclo donde revisa continuamente este bit: mientras siga en 1, la conversión no ha terminado, por lo que el bucle continúa esperando. Cuando la conversión finaliza y el bit ADSC vuelve a cero, se procede a leer el resultado. El ADC entrega el valor dividido en dos registros: primero ADCL (parte baja) y luego ADCH (parte alta). Es obligatorio leer ADCL antes de ADCH para que el valor sea consistente. Los bytes leídos se almacenan en las variables low y high. Por último, en C, el código combina ambos bytes en un entero de 16 bits, desplazando la parte alta 8 posiciones hacia la izquierda y realizando una operación OR con la parte baja. El número resultante, entre 0 y 1023, es la lectura digital del voltaje presente en el pin analógico, y se devuelve al programa.

### **Análisis de comparativo de antes y después**

Se realizó una medición de la mejora de tiempo utilizando la función `micros()`, la cual devuelve una medida en microsegundos del tiempo que lleva el programa ejecutándose, este permite calcular el tiempo que tarda en funcionar tanto el `analogRead()` de la versión sin ensamblador, como la `analogReadAsm()` de la versión optimizada, una vez medido se obtuvo en promedio una mejora de 10 microsegundos la cual equivale a 160 ciclos de reloj, lo cual representa una mejora con la versión inicial.





## Bibliografía

<http://www.dissidents.com/resources/EmbeddedControllers.pdf>

Sensor Kit X 40 (n.d.). <https://sensorkit.joy-it.net/en/sensors/ky-028> KY-028 Temperature sensor (Thermistor)

Arduino Official Store. (n.d.). <https://store.arduino.cc/products/arduino-starter-kit-multi-language>

Aguirre, C. (2024, January 29). *Sensores de temperatura KY-028 Y KY-013*. UNIT Electronics. <https://blog.uelectronics.com/tarjetas-desarrollo/sensores-de-temperatura-ky-028-y-ky-013/>

Arduino. (n.d.). *Uno R3*. docs.arduino.cc. [https://docs.arduino.cc/hardware/uno-rev3/?\\_x\\_tr\\_hist=true#features](https://docs.arduino.cc/hardware/uno-rev3/?_x_tr_hist=true#features)

EEPOWER. (n.d.). *NTC thermistor | Resistor types | resistor guide*. <https://eepower.com/resistor-guide/resistor-types/ntc-thermistor/>

JIMBLOM. (n.d.). *Analog vs. Digital*. Analog vs. Digital - SparkFun Learn. <https://learn.sparkfun.com/tutorials/analog-vs-digital/analog-and-digital-circuits>

Rao, Sri. T. L., M.Tech, Jyoshna, K., Lokesh, A., Sruthi, D., & Jeevansai, N. (2025, May 10). *Smart environment and safety monitoring system using Arduino*. International Journal of Engineering Research & Technology. <https://www.ijert.org/smart-environment-and-safety-monitoring-system-using-arduino>

UNIT Electronics. (2025, August 12). *Modulo KY-028 sensor de Temperatura Digital*.

**[https://uelectronics.com/producto/sensor-de-temperatura-digital-ky-028/?srsltid=AfmBOop9szLQheMSXMg-yKomlmnnp0WaC-89EbQ2hS\\_fyvTamKYQ-x7i](https://uelectronics.com/producto/sensor-de-temperatura-digital-ky-028/?srsltid=AfmBOop9szLQheMSXMg-yKomlmnnp0WaC-89EbQ2hS_fyvTamKYQ-x7i)**

**Wikimedia Foundation. (2024, March 20). *Atmega328*. Wikipedia.  
<https://es.wikipedia.org/wiki/Atmega328>**